Janvier 2026

SIMULATEUR D'UNE MACHINE VIRTUELLE

Guide développeur



Yehor Chernyk & Tom Jougier

TABLE DES MATIERES

1 Vue d'ensemble 3
1.1 Objectif 3
1.2 Architecture de la machine virtuelle 3
1.3 Structure du code source 3
2 Cheminement du développement 4
2.1 Décomposition du problème 4
2.2 Difficultés rencontrées 4
2.3 Utilisation de l'IA 5
3 Spécificités techniques 5
3.1 Encodage des instructions 5
3.2 Extension de signe 6
3.3 Instruction `in` 6
4 Points importants du code 6
4.1 Assembleur : Gestion des étiquettes 6
4.2 Assembleur : Analyse syntaxique des instructions 6
4.3 Processeur : Comportement indéfini 7
5 Portabilité et compatibilité 7
5.1 Compilation 7
5.2 Dépendances 7
6 Fichiers de test 8

CC BY NC SA

1 Vue d'ensemble

1.1 Objectif

Ce projet implémente un simulateur de machine virtuelle capable d'exécuter des programmes écrits en langage assembleur. Le simulateur se compose de deux parties principales :

- Un assembleur qui traduit le code assembleur en langage machine hexadécimal
- Un processeur qui exécute le code machine instruction par instruction

1.2 Architecture de la machine virtuelle

La mémoire est un tableau de caractères : `char memory[65536]`
Les registres généraux sont un tableau d'entiers : `int r[32]`
PC est un entier : `int PC`
Z, N et C sont des entiers : `int Z, N, C`
L'ensemble de ces variables sont déclarées comme variables globales.

Deux structures sont utilisées :

```
typedef struct
{
    char name[64];
    int address;
} Tag;
```

```
typedef struct
{
    int opcode;
    int rd;
    int rn;
    int is_imm;
    int rm;
    int imm_val;
} Instruction;
```

1.3 Structure du code source

Le projet est composé de 6 fichiers :

- `main.c` : point d'entrée, gestion des arguments
- `assembleur.c` : vérification syntaxique et traduction assembleur → machine
- `processeur.c` : exécution des instructions

- `assembleur.h` : header pour la compilation
- `processeur.h` : header pour la compilation
- `Makefile` : script de compilation, nettoyage du répertoire avec `make clean`

2 Cheminement du développement

2.1 Décomposition du problème

Le développement a été découpé en plusieurs phases. La liste suivante suit l'ordre d'exécution :

- Assembleur : Gestion des étiquettes, analyse syntaxique du fichier, traduction du code dans `hexa.txt`
- Chargement en mémoire : Lecture du fichier `hexa.txt` et stockage en mémoire sous forme d'octets
- Décodage des instructions : Extraction des champs (opcode, registres, valeur immédiate)
- Exécution : Implémentation de chaque instruction avec mise à jour des registres et flags.

Les deux parties (assembleur et processeur) ont été soumis à des tests détaillés dans la section 0.

2.2 Difficultés rencontrées

Gestion des étiquettes

Beaucoup d'aspects à prendre en compte : choix d'implémentation, caractères autorisés. Nous avons dû créer plus de fonctions que nous ne l'aurions imaginé (gestion du ' :', vérification de l'étiquette, doublons, remplacement par l'adresse, etc.).

Encodage des instructions

Le plus difficile a été la gestion de l'ensemble des cas possibles, car chaque instruction peut avoir différents formats (distinction entre registres et valeurs immédiates, extension de signe, etc.).

Manipulation des bits

Nous avons eu recours à des décalages (opérateur <<) ainsi qu'à des opérations bit-à-bit. C'était quelque chose de nouveau et assez technique, notamment avec la mémoire : les instructions (stockées sur 32 bits) sont séparées en 4. Par exemple, les bits 8 et 9 ne sont pas dans la même « case » du tableau et le décalage devient rapidement complexe.

Gestion des flags

Les flags (notamment la retenue) ont été source de plusieurs heures de débogage. Il a également fallu adapter au cas par cas : certaines instructions influent différemment sur les flags (`mul` et `shr` ne modifient pas la retenue de la même manière).

Anticipation des erreurs

Nous avons essayé de dresser une liste de tests la plus exhaustive possible, ce qui a amené beaucoup de questions très spécifiques (par exemple, est-ce qu'on autorise l'écriture mémoire sur des cases contenant des instructions ?).

2.3 Utilisation de l'IA

Nous avons toujours tenté de tout faire par nous-même afin de comprendre l'approche adaptée à chaque problème.

Toutefois, nous avons utilisé l'IA (Claude et Gemini) pour les points suivants :

- Décalages bit-à-bit
- Génération de tests pour l'assembleur et le processeur
- Vérification de l'architecture globale du projet
- Aide au débogage lors des tests des sauts conditionnels

3 Spécificités techniques

3.1 Encodage des instructions

Les instructions ont le format suivant :

Opcode (5 bits)	**Dest (rd) (5 bits)**	**Src1 (rn) (5 bits)**	**Imm (1 bit)**	**Src2 (rm/imm) (16 bits)**
Bits 27-31	Bits 22-26	Bits 17-21	Bit 16	Bits 0-15

Le décodage se fait ainsi :

```
opcode = (instruction >> 27) & 0x1F;
rd = (instruction >> 22) & 0x1F;
rn = (instruction >> 17) & 0x1F;
is_imm = (instruction >> 16) & 0x1;
```

Le ET bit-à-bit se fait avec `0x1F` car cela est représenté par `11111`. Cela permet de garder les 5 bits de poids faible après le décalage avec `>>`.

3.2 Extension de signe

Pour les valeurs immédiates 16 bits, nous avons utilisé l'extension de signe afin de gérer les nombres négatifs :

```
short temp = (short)(instruction & 0xFFFF);
int imm_val = (int)temp;
```

3.3 Instruction `in`

L'instruction `in` est sujette aux limitations de l'arithmétique modulaire. Si l'utilisateur rentre un entier dont la représentation binaire dépasse 32 bits, le flag C est mis à 1 et l'entrée est tronquée au 32e bit de poids faible (on force le résultat en int).
En cas d'amélioration du projet, il peut être intéressant de gérer l'entrée de caractères en modifiant la manière de lire les entrées ou en ajoutant une instruction dédiée.

4 Points importants du code

4.1 Assembleur : Gestion des étiquettes

La table des étiquettes est limitée à 100 entrées. Pour un programme plus long, il faudrait utiliser l'allocation dynamique.

La structure est définie dans `assembleur.h`.

4.2 Assembleur : Analyse syntaxique des instructions

La fonction `clean` supprime les espaces/tabulations/retours superflus. Les commentaires n'étant pas encore supportés, ils passeront le nettoyage et génèreront donc une erreur de syntaxe.

Les étiquettes sont analysées et stockées lors d'un premier passage. Lors du deuxième passage (traduction), elles sont remplacées par l'adresse correspondante (numéro de ligne × 4) pour les instructions comme `jmp`.

La fonction `is_registry` vérifie si une chaîne de caractères est de la forme `rX`, avec `X` ∈ {0, ..., 31}.

La fonction `is_ImmValue_ok` vérifie que les valeurs immédiates ont un format correct (`#X`, `#-X`, `#hX` mais pas `#-hX`).

La fonction `is_line_ok` vérifie si une ligne est syntaxiquement correcte.

La fonction `atohex_line` identifie l'opcode, extrait les opérandes et construit l'instruction en hexadécimal.

Dans `assembleur.c`, les fonctions commençant par `ft_` proviennent d'un « bloc » d'un autre projet, et ne sont donc pas optimisées pour les spécificités du présent projet. Elles servent à changer de base pour des conversions.

4.3 Processeur : Comportement indéfini

Lorsque certains problèmes rencontrés sont dus à une erreur de la part du programmeur, le comportement n'est pas toujours intuitif :

- Division par 0 : le registre de destination est mis à 0 et C à 1
- Si on demande à générer un nombre aléatoire dans {a ; … ; b} avec b<a, cela renvoie b

Lorsqu'une erreur survient pendant l'exécution, le programme s'arrête forcément, mais ne donne pas d'informations. Parfois 0 est renvoyé, parfois rien du tout.
En cas d'amélioration du simulateur, il s'agit de l'une des premières modifications à apporter car cela facilite grandement le débogage.

5 Portabilité et compatibilité

5.1 Compilation

La compilation est détaillée dans *user.pdf*.

Privilégiez Linux, MacOS et Windows.

5.2 Dépendances

Le projet n'a que des dépendances standard : `stdio.h`, `stdlib.h`, `string.h`, `ctype.h`, `time.h`, `limits.h` et les headers `assembleur.h` et `processeur.h`

Aucune bibliothèque externe n'est requise.

6 Fichiers de test

Le dossier `resources` contient l'ensemble des tests qui ont été effectués :

- Le fichier `test.c` est à compiler seul : `gcc -Wall -o test test.c`
  Il est la copie de `assembleur.c` sans la fonction `translateCode`, avec une nouvelle fonction test et un main modifié.
  Lors de son exécution, il va simuler la vérification de 144 lignes de code assembleur (indépendantes les unes aux autres) afin de couvrir le plus de cas syntaxiques possibles.
  Les tests sont regroupés par catégorie dans la fonction `test`, et sont commentés afin de savoir ce qui est testé à chaque fois.

- Le sous-dossier `tests` contient des fichiers `.txt` faisant office de test pour le simulateur. Ils sont à appeler comme le ferait un utilisateur avec son propre programme. Le résultat attendu en sortie est indiqué dans `recapitulatif.md`

L'ensemble des tests ont été effectués sur Windows et Linux.